

Document operating system for RAG

Baza dinamica locala de cunostinte pentru documente, RAG, Obsidian, Graphify si memoria agentilor

2026-07-08

Contents

Rezumat executiv	2
Presupuneri hardware corectate	3
Specificatie hardware minima	3
Specificatie hardware recomandata	3
Principii de design	3
1. Router intai, OCR dupa	3
2. Stocheaza straturi, nu doar chunk-uri	4
3. Cautare hibrida, nu doar vector search	4
4. Promovarea in memorie trebuie controlata	4
5. Precizia bate viteza automatizarii	4
Arhitectura tinta	5
Servicii principale	5
Structura de foldere	5
State machine pentru joburi	5
Registrul de surse	6
sources	6
documents	6
pages	6
chunks	7
jobs	7
Rutarea documentelor	7
Reguli de rutare	7
Output router	7
Conducta de extractie	8
Pass 1: extractie normala	8
Pass 2: OCR si vision selectiv	8
Pass 3: validarea extractiei	8
Strategia OCR si VLM pe un singur GPU	9
Scheduling practic	9
Strategie de modele si unelte	9
Rationament local si control agentic	9
Embedding-uri	9
Reranking	10
Modele vision/document	10
Strategie de chunking	10
Tipuri de chunk	10
Reguli de marime	10

Structura parent-child	11
Retrieval si serviciu de raspuns	11
Pipeline de retrieval	11
Filtre metadata	11
Regula de raspuns	11
Export Obsidian si Graphify	12
Export Obsidian	12
Export Graphify	12
Promovare in memorie	13
Loop de update dinamic	13
Detectare update sursa	13
Flow de update	13
Dashboard si workflow pe endpoint-uri	13
Quality gates	14
Scor de extractie	14
Rapoarte necesare	14
Golden test set	14
Securitate si confidentialitate	15
Roadmap de implementare	15
Faza 1: Fundatie	15
Faza 2: Rutare si extractie documente	15
Faza 3: OCR si validare vizuala	16
Faza 4: Indexare si RAG	16
Faza 5: Export Obsidian si Graphify	16
Faza 6: Update-uri dinamice	16
Faza 7: Evaluation harness	17
Stack recomandat pentru primul build	17
Strategie de lucru pentru AI-PC	17
Tinta finala	18
Surse verificate	18

Rezumat executiv

Acest plan descrie un **Document operating system for RAG**, adica un sistem local-first pentru ingestie, extragere, validare, indexare si promovare controlata a documentelor in RAG, Obsidian, Graphify si memoria agentilor.

Scopul nu este doar sa incarci fisiere intr-o baza vectoriala. O baza vectoriala este doar un strat. Tinta corecta este o conducta controlata care pastreaza separat sursa bruta, textul extras, Markdown, JSON, imaginile paginilor, tabelele, figurile, chunk-urile, embedding-urile, indexul full-text, nodurile de graph, notitele Obsidian si deciziile de promovare in memorie.

Fluxul dorit este:

material sursa

- > registru de surse
- > router de documente
- > conducta de extractie
- > OCR / validare vision doar unde este nevoie
- > scoruri de calitate
- > Markdown si JSON structurat
- > chunking cu provenienta pe pagina
- > index de cautare hibrida
- > serviciu RAG cu citari

- > promovare revizuita in Obsidian / Graphify / memorie
- > loop de update pentru surse schimbatoare

Regula principala este: **nu pierde niciodata provenienta**. Orice raspuns dat din KB trebuie sa poata fi urmarit inapoi la fisierul sursa, pagina, chunk, metoda de extractie, timestamp si scor de incredere.

Presupuneri hardware corectate

Masina activa de procesare este un singur AI-PC cu:

- un singur GPU activ: **RTX 5060 Ti 16 GB**
- CPU: **Intel i7-10750 class**
- RAM: **32 GB DDR4**
- laptopurile sunt doar endpoint-uri pentru vizualizare, upload, verificare si control

Nu proiecta sistemul pentru GPU-uri nefolosite. Nu presupune multi-GPU scheduling. Sistemul trebuie sa ruleze cu o singura coada pentru GPU si o coada separata pentru extractie CPU/disk.

Specificatie hardware minima

Aceasta este specificatia minima practica pentru prima versiune folosibila:

- CPU 6-core / 12-thread sau mai bun
- 32 GB RAM
- GPU NVIDIA cu 12 GB VRAM sau mai mult
- SSD 1 TB pentru artefacte active
- stocare separata mare pentru documentele brute si rezultatele generate
- Linux host cu Docker, Python si drivere NVIDIA stabile

AI-PC-ul actual intra in aceasta clasa, mai ales datorita placii cu 16 GB VRAM. Cei 32 GB RAM sunt suficienti pentru inceput, dar batch-urile trebuie controlate.

Specificatie hardware recomandata

Aceasta este tinta recomandata daca sistemul creste:

- CPU cu 8-12 core-uri sau mai bun
- 64 GB RAM
- GPU NVIDIA cu 16 GB VRAM sau mai mult
- NVMe 2 TB pentru procesare activa
- 4 TB+ SSD/HDD pentru biblioteca bruta, imagini de pagina si arhive de artefacte
- tinta de backup zilnic
- UPS daca ruleaza joburi lungi de OCR/indexare

Primul upgrade logic ar fi **RAM la 64 GB**, nu inca un GPU. PDF-urile mari, randarea paginilor, OCR-ul, batch-urile de embedding, Qdrant si dashboard-ul pot presa 32 GB inainte sa fie saturat GPU-ul.

Principii de design

1. Router intai, OCR dupa

Nu face OCR pe fiecare PDF. Este lent si poate strica precizia atunci cand textul embedded este deja bun.

Fiecare fisier trebuie sa treaca intai printr-un router de documente care decide:

- este PDF digital?
- este PDF scanat?
- contine tabele?
- contine formule?
- este multi-coloana?
- contine grafice, figuri sau screenshot-uri?

- este romana, engleza sau mixt?
- este material de universitate, carte, articol, RSS sau captura web?
- are nevoie de fallback?

Apoi sistemul alege cea mai ieftina metoda de extractie care ramane fiabila.

2. Stocheaza straturi, nu doar chunk-uri

Sistemul nu trebuie sa pastreze doar chunk-uri vectoriale. Trebuie sa pastreze:

- fisierul brut
- metadatele sursei
- textul extras
- Markdown extras
- imaginile paginilor
- output OCR
- layout JSON
- tabele JSON / CSV
- descrieri de figuri
- chunk-uri
- embedding-uri
- randuri full-text search
- noduri si muchii de graph
- notite Obsidian
- decizii de promovare in memorie

Vectorii sunt regenerabili. Sursele brute si artefactele de extractie nu sunt regenerabile fara cost si risc.

3. Cautare hibrida, nu doar vector search

Vector search este bun pentru similaritate semantica, dar este slab pentru fapte exacte, coduri de module, formulare juridica, comenzi, referinte de pagina, etichete de tabel si termeni tehnici rari.

Foloseste cautare hibrida:

query

- > keyword / BM25 / SQLite FTS
- > dense vector search
- > optional sparse vector search
- > filtre metadata
- > reranker
- > raspuns cu citari de sursa

4. Promovarea in memorie trebuie controlata

Nu impinge automat fiecare propozitie in Obsidian, Graphify sau memoria lunga a agentului.

Foloseste niveluri de promovare:

- L0 evidenta bruta
- L1 artefacte extrase din document
- L2 chunk-uri indexate
- L3 notite Obsidian curate
- L4 concepte si relatii Graphify
- L5 memorie stabila pentru agent

Doar L3-L5 trebuie tratate ca knowledge pe termen lung. Restul sunt materiale de evidenta si retrieval.

5. Precizia bate viteza automatizarii

Sistemul trebuie sa prefere procesare mai lenta, dar corecta, fata de extractie rapida si rupta. Fiecare pass de extractie trebuie sa produca scor de incredere si raport de esec.

Arhitectura tinta

Servicii principale

Foloseste un numar mic de servicii clare:

- **ingest-api**: primeste fisiere, URL-uri, feed-uri RSS si joburi manuale
- **registry-db**: SQLite pentru surse, documente, pagini, joburi, chunk-uri si scoruri
- **worker-extract**: parser CPU pentru documente digitale
- **worker-ocr-vlm**: worker OCR/vision blocat pe GPU
- **worker-embed**: worker pentru embedding-uri
- **qdrant**: baza vectoriala
- **fts-index**: index full-text SQLite FTS5
- **rag-api**: serviciu de retrieval si raspuns
- **exporter-obsidian**: generator de notite Markdown
- **exporter-graphify**: generator de noduri si relatii
- **review-ui**: coada pentru pagini slabe si promovari
- **scheduler**: systemd timers pentru RSS, fisiere schimbate si reindexare

Structura de foldere

```
/srv/ai-workdesk/kb/  
  raw/  
    books/  
    magazines/  
    university/  
    rss/  
    web/  
    images/  
  registry/  
    kb.sqlite  
  artefacts/  
    extracted_text/  
    extracted_markdown/  
    page_images/  
    ocr_json/  
    layout_json/  
    tables/  
    figures/  
    chunks/  
    reports/  
  indexes/  
    qdrant/  
    sqlite_fts/  
  exports/  
    obsidian/  
    graphify/  
    memory_candidates/  
  eval/  
    golden_docs/  
    extraction_scores/  
    retrieval_scores/
```

State machine pentru joburi

Fiecare sursa trebuie sa treaca prin stari explicite:

```
REGISTERED  
-> ROUTED  
-> EXTRACTED  
-> VALIDATED  
-> CHUNKED
```

- > EMBEDDED
- > INDEXED
- > REVIEWED
- > PROMOTED
- > AVAILABLE_FOR_RAG

Joburile esuate nu trebuie sa dispara. Ele trebuie mutate in:

NEEDS_RETRY
 NEEDS_SECONDARY_OCR
 NEEDS_MANUAL_REVIEW
 BLOCKED_UNSUPPORTED_FORMAT

Registrul de surse

Registrul este panoul de control. El previne munca duplicata si permite update-uri fiabile.

Tabele recomandate:

sources

source_id
 source_kind: file | rss | url | folder_watch | manual_note
 original_location
 canonical_location
 sha256
 size_bytes
 created_at
 registered_at
 last_checked_at
 licence_status
 privacy_class
 status

documents

doc_id
 source_id
 title
 author
 language
 content_type
 page_count
 parser_selected
 ocr_required
 created_at
 updated_at

pages

page_id
 doc_id
 page_number
 page_hash_text
 page_hash_image
 text_confidence
 layout_confidence
 table_confidence
 ocr_used
 needs_review

chunks

chunk_id
doc_id
page_start
page_end
section_title
chunk_type
text
summary
keywords
entities
embedding_model
source_hash
created_at

jobs

job_id
source_id
doc_id
job_type
state
priority
started_at
finished_at
error_message
retry_count

Rutarea documentelor

Routerul trebuie sa aleaga strategia de extractie inainte de procesare scumpa.

Reguli de rutare

Conditie	Prima actiune	Fallback
PDF digital cu text bun	PyMuPDF / pdfplumber / Docling	Marker sau MinerU
PDF scanat	pipeline OCR	validare VLM pe pagini
PDF tehnic cu formule	MinerU / Marker	validare VLM pe pagini esuate
Revista sau layout complex	Docling / Marker	OCR/VLM pe pagini cu scor mic
Imagine sau screenshot	OCR + descriere VLM	review manual daca scorul e mic
Articol RSS	feed parser + extractie continut	browser snapshot daca pagina e dinamica
Pagina web	trafilatura/readability/browser capture	screenshot + OCR daca este blocata
DOCX/PPTX/XLSX	Docling / Marker / MinerU	conversie LibreOffice ca fallback

Output router

```
{  
  "doc_id": "...",  
  "route": "digital_pdf_structured",  
  "language": "en_ro_mixed",  
  "parser_chain": ["docling", "marker_fallback"],  
  "ocr_policy": "selective",  
}
```

```

"vlm_policy": "low_confidence_pages_only",
"expected_risk": "tables_and_two_column_layout"
}

```

Conducta de extractie

Pass 1: extractie normala

Pentru PDF-uri digitale, incepe cu extractia normala:

- text embedded
- headings
- numere de pagina
- tabele
- figuri
- captions
- linkuri
- referinte
- reading order
- formule unde este posibil

Unelte candidate:

- PyMuPDF pentru text rapid si metadata de pagina
- pdfplumber pentru inspectie text si tabele
- Docling pentru conversie structurata de documente
- Marker pentru Markdown/JSON/chunks
- MinerU pentru documente tehnice si stiintifice

Pass 2: OCR si vision selectiv

Ruleaza OCR doar unde este nevoie:

- pagina nu are text embedded
- textul embedded este corupt
- textul romanesc sau englezesc este rupt
- scorul tabelului este mic
- extractia formulelor este slaba
- pagina are screenshot-uri, diagrame, grafice sau scris de mana
- reading order pare gresit

Unelte/modele candidate:

- PaddleOCR / PaddleOCR-VL pentru OCR multilingv si parsing de documente
- olmOCR pentru documente PDF/imagine unde reading order conteaza
- Qwen2.5-VL sau Qwen3-VL class model pentru validare pe pagini selectate

Pentru ca AI-PC-ul are un singur GPU de 16 GB, toate joburile OCR/VLM trebuie sa foloseasca un **GPU lock**. Un singur job GPU greu trebuie sa ruleze la un moment dat.

Pass 3: validarea extractiei

Fiecare pagina primeste scor:

```

{
  "page_number": 12,
  "text_confidence": 0.94,
  "layout_confidence": 0.88,
  "table_confidence": 0.72,
  "ocr_used": true,
  "needs_review": false,
  "warnings": ["possible_table_loss"]
}

```

Verificari:

- pagina goala sau aproape goala
- eliminare header/footer repetat
- reading order gresit
- cuvinte rupte intre randuri
- sanity check pentru randuri/coloane de tabel
- verificare formule
- detectare imagine/figura
- mismatch de limba
- pagini duplicate

Strategia OCR si VLM pe un singur GPU

GPU-ul de 16 GB este suficient pentru o conducta locala serioasa, dar numai daca joburile sunt puse in coada.

Regula recomandata:

Extractia CPU poate rula in paralel.

OCR/VLM pe GPU ruleaza un singur job la un moment dat.

Batch-urile de embedding ruleaza cand OCR/VLM este idle.

Modelul de chat/retrieval primeste prioritate peste procesarea de fundal.

Scheduling practic

Foloseste trei cozi:

- **fast queue:** PDF-uri mici digitale, RSS, pagini web
- **batch queue:** carti, reviste, materiale mari de universitate
- **heavy queue:** PDF-uri scanate, OCR, validare VLM, reparare tabele

Politica exemplu:

08:00-22:00: ingestie usoara si prioritate retrieval

22:00-07:00: batch OCR, embedding-uri, rebuild de index

Daca sistemul este folosit pentru studiu sau coding, opreste OCR-ul greu.

Strategie de modele si unelte

Rationament local si control agentic

Candidate recomandate:

- Qwen3 14B quantized prin Ollama pentru rationament local principal
- Qwen3 8B pentru clasificare si rutare mai rapida
- Qwen2.5-Coder 14B sau similar pentru reparatii de cod si scripturi de pipeline

Foloseste LLM-ul pentru decizii, nu pentru extractie bruta. Parser-ele si OCR-ul trebuie sa extraga. LLM-ul trebuie sa aleaga ruta, sa valideze sumarizari, sa creeze metadata si sa produca candidati de memorie.

Embedding-uri

Recomandare principala:

- **BGE-M3** pentru retrieval multilingv, mai ales material mixt engleza/romana

De ce:

- multilingv
- dense retrieval
- sparse retrieval
- multi-vector retrieval
- suporta granularitate mai lunga decat multe modele mici de embedding

Candidat viitor:

- **Qwen3 Embedding 0.6B / 4B / 8B**, testat dupa ce conducta de baza merge

Nu schimba modelele de embedding la intamplare. Stocheaza numele si versiunea modelului de embedding pentru fiecare chunk.

Reranking

Adauga reranking dupa ce retrieval-ul hibrid de baza merge.

Candidati:

- familia BGE reranker
- familia Qwen3 reranker

Reranking-ul trebuie sa fie optional pentru ca adauga latentia. Foloseste-l pentru raspunsuri finale, nu pentru fiecare cautare exploratorie.

Modele vision/document

Candidate:

- PaddleOCR / PaddleOCR-VL pentru OCR si document parsing multilingv
- olmOCR pentru documente PDF/imagine unde reading order conteaza
- Qwen2.5-VL sau Qwen3-VL class model pentru review vizual al paginilor cu scor mic

Nu pune un VLM sa proceseze carti intregi pagina cu pagina daca parser-ul normal nu a esuat. VLM-urile sunt mai scumpe si mai lente.

Strategie de chunking

Chunking-ul trebuie sa pastreze sensul si evidenta pe pagina.

Tipuri de chunk

Foloseste tipuri diferite:

- definitie
- explicatie
- procedura
- tabel
- figura
- cod
- citat
- formula
- sumar
- referinta
- intrebare-raspuns
- nota de curs / assignment

Reguli de marime

Tip sursa	Regula de chunking
Note de universitate	pe heading, 400-900 cuvinte
Carti	pe sectiune, 700-1200 cuvinte
Reviste	pe articol/sectiune
Tabele	un tabel = un chunk plus sumar
Cod/comenzi	pastreaza intact
Diagrame	text OCR + caption + imagine pagina legata
RSS/web	sectiuni de articol cu URL canonic
Definitii	chunk-uri atomice separate

Structura parent-child

Foloseste un arbore:

```
document
  chapter
    section
      chunk
        table
        figure
        quote
```

Astfel RAG-ul poate gasi un chunk mic, dar poate extinde contextul la sectiunea parinte cand este nevoie.

Retrieval si serviciu de raspuns

Pipeline de retrieval

```
query utilizator
-> detectare limba
-> normalizare / query expansion
-> filtre metadata
-> SQLite FTS / BM25 search
-> dense vector search
-> optional sparse vector search
-> fusion rezultate
-> reranking
-> context pack
-> raspuns cu citari
```

Filtre metadata

Suporta filtre precum:

- tip sursa: universitate, carte, RSS, web, imagine
- cod modul
- autor
- an
- limba
- prag de incredere
- data ingestiei
- doar reviewed
- include/exclude pagini doar OCR
- clasificare private/public

Regula de raspuns

Serviciul de raspuns nu trebuie sa spuna ca ceva vine din KB daca nu poate cita sursa.

Fiecare raspuns generat trebuie sa includa:

Surse:

- titlu document
- numar pagina sau URL articol
- chunk ID
- confidence de extractie

Daca retrieval-ul este slab, raspunsul trebuie sa spuna:

Am gasit material apropiat, dar nu pot confirma raspunsul din sursele indexate.

Export Obsidian si Graphify

Export Obsidian

Obsidian trebuie sa primeasca notite curate, nu dump brut.

Tipuri de notite recomandate:

- notite de sursa
- notite de concept
- notite de modul
- harti de topic
- definitii
- suport pentru assignment
- sumare saptamanale de universitate
- sumare de lectura
- intrebari nerezolvate

Exemplu frontmatter:

```
title: OAuth 2.0 Authorization Code Flow
type: concept
source_count: 3
confidence: reviewed
created: 2026-07-08
updated: 2026-07-08
tags:
  - cybersecurity
  - authentication
  - university
```

Exemplu corp de nota:

```
# OAuth 2.0 Authorization Code Flow

## Summary
Sumar scurt pentru om.

## Key points
- Punct 1
- Punct 2

## Source evidence
- COM4010 Lecture 04, page 22, chunk abc123
- Security textbook, page 118, chunk def456

## Related
- [/OpenID Connect/]
- [/Access Token/]
- [/Refresh Token/]
```

Export Graphify

Graphify trebuie sa primeasca entitati si relatii:

```
{
  "node_type": "concept",
  "name": "OAuth 2.0 Authorization Code Flow",
  "aliases": ["auth code flow"],
  "sources": [
    {"doc_id": "...", "page": 22, "chunk_id": "abc123"}
  ],
  "relations": [
    {"type": "used_in", "target": "Web authentication"},
    {"type": "contrasts_with", "target": "Implicit Flow"}
  ]
}
```

```
]
}
```

Promovare in memorie

Promoveaza in memoria lunga a agentului doar cunostinte stabile si reutilizabile:

- reguli de proiect
- preferinte recurente
- definitii stabile
- sumare de curs/modul
- workflow-uri validate
- decizii de arhitectura

Nu promova:

- OCR neverificat
- headline-uri RSS temporare
- chunk-uri duplicate
- sumare zgomotoase
- extractie cu scor mic
- continut complet din carti protejate de copyright

Loop de update dinamic

Detectare update sursa

Sursa	Metoda de detectare
RSS	ETag, Last-Modified, URL hash
Pagina web	URL canonic + content hash
PDF fisier	file hash + page hash
Folder universitate	file watcher + hash check
Carte	de obicei static, fara check recurent
GitHub repo	commit hash
Nota Obsidian	frontmatter hash

Flow de update

check programat

- > detecteaza sursa schimbata
- > compara hash-uri
- > extrage doar pagini/articole schimbate
- > reconstruieste chunk-urile afectate
- > sterge vectorii vechi pentru chunk-uri schimbate
- > insereaza vectori noi
- > actualizeaza indexul FTS
- > actualizeaza candidatii Graphify
- > actualizeaza candidatii Obsidian
- > genereaza raport

Foloseste systemd timers la inceput. n8n poate fi adaugat mai tarziu pentru workflow vizual, aprobari si notificari, dar nu ar trebui sa fie motorul principal de procesare.

Dashboard si workflow pe endpoint-uri

Laptopurile trebuie sa fie doar endpoint-uri.

Workflow recomandat:

1. Deschizi dashboard-ul de pe laptop.

2. Arunci fisier, folder, RSS feed sau URL.
3. Alegi categoria sursei daca este nevoie.
4. Sistemul inregistreaza sursa si porneste rutarea.
5. Dashboard-ul arata statusul de extractie.
6. Paginile cu scor mic apar in coada de review.
7. Aprobi sau respingi candidatii de promovare.
8. Notitele aprobate merg in Obsidian si Graphify.
9. Endpoint-ul RAG raspunde cu citari.

Carduri utile in dashboard:

- coada de ingestie
- coada OCR
- pagini esuate
- tabele cu scor mic
- surse indexate recent
- candidati export Obsidian
- candidati export Graphify
- rezultate teste retrieval
- utilizare stocare
- status coada GPU

Quality gates

Scor de extractie

Fiecare document trebuie sa primeasca:

- scor text
- scor layout
- scor tabel
- scor figura
- scor citare/provenienta
- confidence OCR
- confidence limba
- status final pass/fail

Rapoarte necesare

Pentru fiecare batch:

extraction_report.md
low_confidence_pages.md
failed_tables.json
changed_sources.json
retrieval_eval.json
promotion_candidates.md

Golden test set

Creeaza o biblioteca mica de benchmark:

- 10 PDF-uri digitale
- 10 PDF-uri scanate
- 5 documente cu multe tabele
- 5 documente in romana
- 5 documente in engleza
- 5 documente mixte romana/engleza
- 5 PDF-uri de universitate
- 5 pagini cu diagrame sau screenshot-uri
- 5 surse RSS/web

Verificari manuale:

- numar pagini corect
- textul nu lipseste
- reading order corect
- randuri si coloane de tabel pastrate
- figuri detectate
- formule acceptabile
- cautarea returneaza sursa corecta in top 5
- raspunsul citeaza pagina corecta

Securitate si confidentialitate

Trateaza toate documentele ca input nevalidat.

Reguli:

- proceseaza fisiere in continere unde este posibil
- dezactiveaza executia de macro-uri
- nu rula automat scripturi embedded
- pastreaza fisierele brute read-only dupa inregistrare
- tine documentele private local
- evita loguri cu text complet sensibil
- clasifica sursele ca private, course, project, public sau unknown
- nu trimite documente private in cloud fara aprobare explicita
- tine backup-urile criptate daca includ materiale personale sau universitare

Roadmap de implementare

Faza 1: Fundatie

Scop: ingestie controlata fara scrieri in memorie.

Construieste:

- structura de foldere
- registru SQLite
- hashing fisiere
- API de inregistrare sursa
- folder de upload
- status simplu in dashboard
- job state machine
- folder de rapoarte

Criterii de iesire:

- fisierele pot fi adaugate
- duplicatele sunt detectate
- joburile sunt urmarite
- fisierele brute raman imutabile

Faza 2: Rutare si extractie documente

Construieste:

- router de documente
- parser PDF digital
- output Markdown/JSON
- randare imagine pagina
- incercare extractie tabele
- raport de extractie

Criterii de iesire:

- PDF-urile digitale produc Markdown si JSON
- fiecare pagina are metadata

- paginile esuate sunt vizibile

Faza 3: OCR si validare vizuala

Construieste:

- fallback OCR
- detectie pagini cu scor mic
- OCR/VLM GPU lock
- coada reparare tabele
- ruta pentru PDF scanat

Criterii de iesire:

- PDF-urile scanate pot fi procesate
- paginile slabe sunt rutate corect
- OCR-ul nu blocheaza retrieval-ul normal

Faza 4: Indexare si RAG

Construieste:

- chunker
- worker embedding BGE-M3
- colectie Qdrant
- index SQLite FTS5
- endpoint retrieval hibrid
- endpoint raspuns cu citari

Criterii de iesire:

- utilizatorul poate pune intrebari
- raspunsurile citeaza document/pagina/chunk
- retrieval-ul slab este raportat onest

Faza 5: Export Obsidian si Graphify

Construieste:

- template-uri Obsidian
- export JSON nod/muchie Graphify
- coada de promovare
- review UI

Criterii de iesire:

- nu exista dump automat in brain
- notitele revizuite pot fi exportate
- relatiile graph citeaza surse

Faza 6: Update-uri dinamice

Construieste:

- RSS checker
- folder watcher
- URL checker
- detector pagini schimbate
- reindexare incrementală
- rapoarte de update

Criterii de iesire:

- doar materialul schimbat este reprocesat
- vectorii vechi sunt eliminati
- rapoartele arata ce s-a schimbat

Faza 7: Evaluation harness

Construieste:

- set de documente golden
- scoruri de extractie
- scoruri de retrieval
- rapoarte comparatie modele
- teste de regresie

Criterii de iesire:

- schimbarile la parsere/modele pot fi masurate
- extractia rupta este detectata inainte de promovare

Stack recomandat pentru primul build

Incepe cu:

Python
SQLite
PyMuPDF
pdfplumber
Docling
Marker
MinerU
PaddleOCR / PaddleOCR-VL
olmOCR
BGE-M3 via FlagEmbedding
Qdrant
SQLite FTS5
Ollama pentru control LLM local
systemd timers
export Markdown Obsidian
export JSON Graphify

Adauga mai tarziu:

n8n pentru aprobari vizuale si notificari
Qwen3 Embedding / reranker dupa ce retrieval-ul de baza merge
Qwen-VL pentru validare de pagina dupa ce ruta OCR este stabila
notificari Telegram / WhatsApp

Strategie de lucru pentru AI-PC

Mod recomandat de operare:

- un singur job GPU greu la un moment dat
- OCR de fundal mai ales noaptea
- batch-uri de embedding dupa extractie
- live RAG/chat are prioritate
- batch size mic pana masori memoria
- nu procesa o carte mare fara checkpoint-uri de batch
- scrie fiecare artefact intermediar pe disk
- pastreaza toate hash-urile si versiunile de model

Politica implicita buna:

Fisiere mici: proceseaza imediat
Carti: proceseaza in batch-uri
Fisiere scanate: coada pentru fereastra OCR
RSS: proceseaza programat
Pagini web: proceseaza cu hash checks
Materiale de universitate: proceseaza imediat, apoi creeaza candidati de review

Tinta finala

Sistemul final trebuie sa iti permita sa arunci un PDF, document scanat, feed RSS, carte, imagine sau material de universitate si AI-PC-ul sa faca automat:

1. inregistrare
2. clasificare
3. extractie
4. validare
5. OCR doar unde este nevoie
6. Markdown si JSON
7. chunking cu provenienta
8. embedding
9. indexare pentru cautare hibrida
10. candidati review pentru Obsidian/Graphify
11. update ulterior pentru surse schimbate
12. raspunsuri cu citari

Ideea de baza este simpla:

Construieste intai un sistem de operare pentru documente, apoi pune RAG peste el.

Baza vectoriala este doar o piesa din stack. Valoarea reala este conducta controlata care transforma material uman dezordonat in knowledge de incredere, legat de surse.

Surse verificate

Verificate la 2026-07-08. Foloseste aceste surse ca punct de plecare, nu ca adevar permanent. Reverifica versiunile inainte de instalare.

- **Docling** - Document parsing and conversion, including PDF understanding, OCR, tables, formulas, and reading order.
<https://www.docling.ai/>
- **Marker** - Converts PDF, images, Office files, EPUB, HTML to Markdown, JSON, chunks, and HTML.
<https://github.com/datalab-to/marker>
- **MinerU** - Document parsing for PDF, image, DOCX, PPTX, XLSX to Markdown and JSON, useful for scientific/technical documents.
<https://github.com/opendatalab/MinerU>
- **PaddleOCR / PaddleOCR-VL** - OCR and document parsing toolkit for PDFs/images, including multilingual document parsing.
<https://github.com/PaddlePaddle/PaddleOCR>
- **olmOCR** - Toolkit for converting PDFs and image-based documents into clean text in natural reading order.
<https://github.com/allenai/olmocr>
- **Qdrant hybrid queries** - Hybrid retrieval combining dense and sparse representations.
<https://qdrant.tech/documentation/search/hybrid-queries/>
- **SQLite FTS5** - SQLite full-text search virtual table module.
<https://www.sqlite.org/fts5.html>
- **FlagEmbedding / BGE-M3** - BGE-M3 multilingual dense, sparse, and multi-vector retrieval.
<https://github.com/FlagOpen/FlagEmbedding>
- **Qwen3 Embedding** - Embedding and reranking model family for multilingual retrieval.
<https://github.com/QwenLM/Qwen3-Embedding>
- **Ollama Qwen3 14B** - Qwen3 14B local model packaging information for Ollama.
<https://ollama.com/library/qwen3:14b>
- **systemd timers** - Timer-based activation for recurring local jobs.
<https://man.archlinux.org/man/systemd.timer.5.en>